

MULTI-SEQUENCE EM WRAPPER FUNCTION FOR TRAINING MODEL WITH MULTIPLE SEQUENCES

This last-minute inclusion to our repo illustrates the invaluable lesson we learned regarding training the Baum-Welch implementation with all three observations such that the model continually improves but does not implement artificial values by concatenating the sequences. In our implementation notebook is our preliminary implementation, in which we were inadvertently training the model on each observation separately, as we did with the original model. Our outputs demonstrate the wildly inconsistent values we received by training this way, and it was at this point that we recognized something must be wrong with our setup. Please see the "Challenges" section in our README for a detailed explanation of this experience and what we learned from it. Thank you for taking the time to acknowledge this file despite it not being in the traditional submissible format. We will be adding this to our notebook, but with limited time to complete the project when we made this discovery, this was our best way of sharing what we learned with you without risking overwriting our preliminary implementation, which is equally important to this story!

```
def baumwelch_multi(self, observations, n_iter=25):
    """
    Multi-sequence Baum-Welch training to train on 3 observations, refining probabilities
    through each iteration
    observations: list of strings (each validated observation sequence)
    """

    # validate all observations
    observations = [self.validate_observation(obs) for obs in observations]

    states = list(self.states)
    symbols = sorted(self.valid_chars)
    num_states = len(states)

    for _ in range(n_iter):

        # accumulate expected counts across all sequences
        total_log_gamma = None
        total_log_xi = None
```

```

for obs in observations:
    T = len(obs)

    # forward/backward for this sequence
    fwd, _ = self.forward_algorithm(obs)
    bwd = self.backward_algorithm(obs)

    # compute log-gamma for this sequence
    log_gamma = np.zeros((num_states, T), dtype=np.float64)
    for t in range(T):
        log_norm_t = np.logaddexp.reduce(fwd[:, t] + bwd[:, t])
        for i in range(num_states):
            log_gamma[i, t] = fwd[i, t] + bwd[i, t] - log_norm_t

    # compute log-xi for this sequence
    log_xi = np.zeros((num_states, num_states, T - 1), dtype=np.float64)
    for t in range(T - 1):
        next_char = obs[t + 1]
        emit_next = np.array([self.emit_matrix[si, next_char] for si in states])

        for i, si in enumerate(states):
            for j, sj in enumerate(states):
                log_xi[i, j, t] = (
                    fwd[i, t]
                    + self.trans_matrix[si, sj]
                    + emit_next[j]
                    + bwd[j, t + 1]
                )

        log_norm_t = np.logaddexp.reduce(log_xi[:, :, t].ravel())
        log_xi[:, :, t] -= log_norm_t

    # accumulate counts
    if total_log_gamma is None:
        total_log_gamma = log_gamma
        total_log_xi = log_xi
    else:
        # logaddexp for accumulation
        total_log_gamma = np.logaddexp(total_log_gamma, log_gamma)

```

```

total_log_xi = np.logaddexp(total_log_xi, log_xi)

# M-step: update initial probabilities
new_initial_probs = {}
log_gamma_t0 = total_log_gamma[:, 0]
log_norm = np.logaddexp.reduce(log_gamma_t0)
for i, si in enumerate(states):
    val = log_gamma_t0[i] - log_norm
    if np.isneginf(val):
        val = np.log(1e-12)
    new_initial_probs[si] = float(np.exp(val))

# M-step: update transition probabilities
new_transition_probs = {}
for i, si in enumerate(states):
    new_transition_probs[si] = {}
    log_denom = np.logaddexp.reduce(total_log_gamma[i, :-1])

    for j, sj in enumerate(states):
        log_numer = np.logaddexp.reduce(total_log_xi[i, j, :])
        if np.isneginf(log_numer):
            log_numer = np.log(1e-12)
        new_transition_probs[si][sj] = float(np.exp(log_numer - log_denom))

# M-step: update emission probabilities
new_emission_probs = {}
for i, si in enumerate(states):
    new_emission_probs[si] = {}
    log_denom = np.logaddexp.reduce(total_log_gamma[i, :])

    for sym in symbols:
        mask = []
        for obs in observations:
            mask.extend([t for t in range(len(obs)) if obs[t] == sym])

        if mask:
            log_numer = np.logaddexp.reduce(total_log_gamma[i, mask])
        else:
            log_numer = np.log(1e-12)

```

```

        new_emission_probs[si][sym] = float(np.exp(log_numer - log_denom))

    # update model parameters
    self.initial_probs = new_initial_probs
    self.transition_probs = new_transition_probs
    self.emission_probs = new_emission_probs
    self.trans_matrix = StrMatrix(self.transition_probs)
    self.emit_matrix = StrMatrix(self.emission_probs)

    return new_initial_probs, new_transition_probs, new_emission_probs

```

DISPLAY RESULTS OF MULTI-SEQUENCE EM

```

# Display trained multi-sequence model results for obs1
print("=====> * TRAINED MULTI-SEQUENCE MODEL - OBS1 * <=====")
print("Viterbi Path:", path1_t)
display(show_matrix_html(format_matrix(vmat1_t, trained_model.states, obs1)))
print("Log Likelihood:", log_likelihood1_t)
print("Most Likely States:", most_likely_states1_t)
display(show_matrix_html(format_matrix(fb_matrix1_t, trained_model.states, obs1)))
display(HTML("<br>"))

# Display trained multi-sequence model results for obs2
print("=====> * TRAINED MULTI-SEQUENCE MODEL - OBS2 * <=====")
print("Viterbi Path:", path2_t)
display(show_matrix_html(format_matrix(vmat2_t, trained_model.states, obs2)))
print("Log Likelihood:", log_likelihood2_t)
print("Most Likely States:", most_likely_states2_t)
display(show_matrix_html(format_matrix(fb_matrix2_t, trained_model.states, obs2)))
display(HTML("<br>"))

# Display trained multi-sequence model results for obs3
print("=====> * TRAINED MULTI-SEQUENCE MODEL - OBS3 * <=====")
print("Viterbi Path:", path3_t)
display(show_matrix_html(format_matrix(vmat3_t, trained_model.states, obs3)))
print("Log Likelihood:", log_likelihood3_t)

```

```
print("Most Likely States:", most_likely_states3_t)
display(show_matrix_html(format_matrix(fb_matrix3_t, trained_model.states, obs3)))
```

RESULTS OF MULTI-SEQUENCE EM TRAINED MODEL

=====> * TRAINED MULTI-SEQUENCE MODEL - OBS1 * <=====

Viterbi Path: ['H', 'H', 'H', 'L', 'L', 'L', 'L', 'L', 'L']

Log Likelihood: -12.483820056859475

Most Likely States: ['H' 'H' 'H' 'L' 'H' 'L' 'H' 'L' 'L']

=====> * TRAINED MULTI-SEQUENCE MODEL - OBS2 * <=====

Viterbi Path: ['L', 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L']

Log Likelihood: -12.483043753806445

Most Likely States: ['L' 'L' 'H' 'H' 'L' 'L' 'L' 'H' 'H']

=====> * TRAINED MULTI-SEQUENCE MODEL - OBS3 * <=====

Viterbi Path: ['L', 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L']

Log Likelihood: -12.490497134887951

Most Likely States: ['L' 'L' 'L' 'H' 'H' 'H' 'L' 'H' 'L']

RESULTS FROM ORIGINAL MODEL FOR EACH OBSERVATION

=====> * ORIGINAL MODEL - OBS1 * <=====

Viterbi Path: ['H', 'H', 'H', 'L', 'L', 'L', 'L', 'L', 'L']

Log Likelihood: -12.483820056859475

Most Likely States: ['H' 'H' 'H' 'L' 'H' 'L' 'H' 'L' 'L']

=====> * ORIGINAL MODEL - OBS2 * <=====

Viterbi Path: ['L', 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L']

Log Likelihood: -12.483043753806445

Most Likely States: ['L' 'L' 'H' 'H' 'L' 'L' 'L' 'H' 'H']

=====> * ORIGINAL MODEL - OBS3 * <=====

Viterbi Path: ['L', 'L', 'L', 'L', 'L', 'L', 'L', 'L', 'L']

Log Likelihood: -12.490497134887951

Most Likely States: ['L' 'L' 'L' 'H' 'H' 'H' 'L' 'H' 'L']